ACADEMIC PROJECT REPORT

SMART HEALTHCARE PATIENT MONITORING SYSTEM

IOT SIMULATION & NETWORK COMMUNICATION ANALYSIS

> **ACADEMIC DISCLAIMER & SCOPE:** This project is a pure software-based Internet of Things (IoT) simulation. No physical sensors or real medical hardware devices are connected. All patient data, including heart rate, body temperature, and SpO_2 saturation readings, are synthetically generated for demonstration. This system is designed exclusively for academic evaluation and software networking research and is not intended for medical diagnosis, clinical decisions, or real patient monitoring.

Submitted By:
Srilekha Sunil

Department:
Computer Science / Cyber Security

Project Domain:
Internet of Things (IoT), Healthcare Telemetry, Networking & Cyber Security

Project Type:
Software-Based IoT Simulation Engine

Repository URL:
https://github.com/srilekhasunil65-byte/Simulation_Healthcare_IoT

1. DECLARATION

I, **Sri Lekha**, hereby declare that the academic project report titled **"SMART HEALTHCARE PATIENT MONITORING SYSTEM – IoT SIMULATION"** submitted to the Department of Computer Science / Cyber Security is an authentic record of software simulation work carried out by me. The source code, simulation scripts, and architecture design documented in this project match the repository maintained at *https://github.com/srilekhasunil65-byte/Simulation_Healthcare_IoT*.

I confirm that this project relies entirely on software-based IoT simulation modules and synthetic telemetry algorithms. No live human trials, physical medical sensors, or actual clinical patient monitoring were performed. I have clearly documented tested components alongside experimental modules where testing remains incomplete, adhering to strict academic integrity standards.

SriLekha Sunil
Student Researcher

2. ACKNOWLEDGEMENTS

I express my sincere gratitude to the Department of Computer Science and Cyber Security for providing the computational infrastructure, network simulation tools, and academic guidance required to complete this software project.

I extend my profound thanks to my project guide and faculty mentors for their invaluable feedback during the architecture design, protocol analysis, and debugging stages of the MQTT, Flask, SQLite, CoAP, and Contiki-NG/Cooja simulation frameworks. Their constant encouragement pushed me to explore lightweight messaging protocols and low-power wireless personal area network (6LoWPAN) architectures in depth.

Finally, I express my appreciation to open-source software communities, including the developers of Python, Paho-MQTT, Flask, HiveMQ, aiocoap, and the Contiki-NG project team, whose tools and documentation made this simulation environment possible.

3. ABSTRACT

Modern healthcare infrastructure increasingly relies on Internet of Things (IoT) technologies to provide remote patient telemetry, continuous physiological observation, and real-time alerts. However, deploying physical IoT medical devices presents significant deployment costs, protocol validation challenges, and cybersecurity vulnerabilities. This report presents a complete software-based **Smart Healthcare Patient Monitoring System IoT Simulation** designed to emulate real-time physiological data telemetry, lightweight protocol transmission, data persistence, and web dashboard visualization.

The system leverages a multi-threaded Python engine to generate synthetic real-time telemetry —including Heart Rate (BPM), Body Temperature (°C), and Blood Oxygen Saturation (SpO_2)—for multiple simulated patients. Sensor telemetry is published over the lightweight Message Queuing Telemetry Transport (MQTT) protocol using the Paho-MQTT client library via the HiveMQ public cloud broker. An asynchronous backend subscriber consumes these payloads and commits them to a local SQLite database engine. A Flask-driven web application provides a responsive dashboard with automatic UI refreshing for real-time monitoring.

Additionally, the project investigates constrained networking paradigms by implementing a Constrained Application Protocol (CoAP) server/client framework via *aiocoap* and deploying low-power wireless personal area networks (6LoWPAN / IPv6) within the Contiki-NG OS operating inside the Cooja simulation environment. Full operational success is demonstrated across the MQTT, SQLite, and Flask dashboard pipelines, while preliminary network activity and server setup are honestly documented for CoAP and Contiki-NG/Cooja environments.

4. TABLE OF CONTENTS

1. Cover Page & Metadata 1

2. Declaration & Acknowledgements 2

3. Abstract & Table of Contents 3

4. Executive Introduction 4

5. Problem Statement & Research Objectives 4

6. Existing vs. Proposed Architecture 5

7. Detailed Technology Stack 6

8. End-to-End System Architecture 7

9. Detailed System Modules 8

10. Patient Telemetry Data Simulation Engine 9

11. MQTT Messaging Protocol Implementation 10

12. Flask Dashboard & SQLite Database Architecture 11

13. CoAP & Contiki-NG / Cooja Simulation 12

14. Testing Results, Limitations & GitHub Repository Structure 13

15. Future Enhancements, Conclusion & References 14

5. INTRODUCTION

The integration of the Internet of Things (IoT) into modern clinical workflows has revolutionized digital health monitoring. By deploying networked sensing devices near or on the human body, medical staff can continuously track vital physiological metrics without requiring continuous manual checks. However, physical deployment of healthcare IoT nodes involves strict regulatory compliance, high hardware costs, and significant operational risks during early protocol design and testing phases.

This project addresses these hardware constraints by establishing a flexible, pure software-based IoT simulation framework. Built using Python, Flask, MQTT, and SQLite, the application emulates an ecosystem of patient monitoring nodes transmitting critical metrics to a centralized monitoring server. The system demonstrates the end-to-end data pipeline: from synthetic data generation, publish-subscribe networking over public cloud brokers, and relational data storage, to dynamic web dashboard visualization.

6. PROBLEM STATEMENT

Developing healthcare IoT systems directly on physical hardware faces several critical bottlenecks:

- **Hardware Accessibility & Cost:** Acquiring medical-grade biological sensors and microcontrollers for early software prototyping is expensive and resource-intensive.
- **Protocol Selection Complexity:** Choosing between TCP-based MQTT, UDP-based CoAP, and 6LoWPAN mesh protocols requires controlled test environments before committing to physical node deployments.
- **Risk of System Failure:** Unchecked code defects or unhandled network latency can compromise vital telemetry during live physical testing.

Thus, there is a clear academic and engineering need for a robust, open-source software simulation engine that allows developers to model physiological sensor streams, evaluate publish-subscribe communication pipelines, and benchmark data persistence layers risk-free.

7. PROJECT OBJECTIVES

The primary objectives of this software-based IoT research project are:

1. **Synthesize Multi-Patient Telemetry:** Build a Python multi-patient simulator producing randomized but realistic physiological values (Heart Rate: 60–100 BPM, Temperature: 36.5–37.5°C, SpO_2: 95–100%).
2. **Implement Lightweight Telemetry via MQTT:** Integrate Paho-MQTT to publish structured JSON telemetry to the public HiveMQ broker over standard TCP ports.
3. **Develop Asynchronous Database Persistence:** Construct a multi-threaded backend subscriber that ingests broker messages and logs them into a relational SQLite database.
4. **Create a Web Monitoring Dashboard:** Build a responsive Flask web interface with automatic browser polling to display current patient statistics and alerts.
5. **Explore Constrained & Mesh Protocols:** Set up experimental CoAP servers and Contiki-NG/Cooja 6LoWPAN network simulations to study UDP-based low-power communication.

8. EXISTING SYSTEM ANALYSIS

Traditional patient monitoring systems in legacy hospital environments rely primarily on standalone hardware monitors wired directly to bedside terminals. While highly reliable, these existing architectures exhibit noticeable structural drawbacks:

- **Point-to-Point Hardware Dependency:** Bedside monitors require direct physical connections or proprietary local hospital network gateways, creating isolated silos of patient data.
- **High Network Overhead:** Legacy polling architectures using HTTP/1.1 REST APIs transmit heavy HTTP headers (often 500+ bytes per request), consuming excessive bandwidth and power.
- **Lack of Flexible Protocol Benchmarking:** Proprietary hardware does not easily allow researchers to compare lightweight publish-subscribe protocols (MQTT) against request-response protocols (CoAP).

9. PROPOSED SYSTEM ARCHITECTURE

The proposed **Smart Healthcare Patient Monitoring System** replaces direct hardware reliance with a modular, highly decoupled software simulation architecture. By combining pub/sub messaging with lightweight web visualization, the proposed design delivers several key improvements:

Feature / Aspect	Traditional Existing System	Proposed Software IoT System
Data Transmission	Synchronous HTTP Polling / Wired Cables	Asynchronous Pub/Sub (MQTT over TCP)
Header Overhead	Heavy HTTP Headers (>500 bytes)	Minimal MQTT Fixed Headers (2 bytes)
System Coupling	Tightly coupled hardware and UI	Decoupled via Cloud Broker & SQLite

Feature / Aspect	Traditional Existing System	Proposed Software IoT System
Scalability	Limited by physical sensor ports	Scalable software simulation threads
Cost & Risk	High hardware cost and deployment risk	Zero hardware cost; risk-free prototyping

> **SYSTEM CLARIFICATION:** The proposed system operates entirely in software. Simulated patient data flows asynchronously through lightweight network protocols to demonstrate real-time telemetry processing without physical medical equipment.

10. TECHNOLOGIES USED & SOFTWARE STACK

The system utilizes a stack of open-source programming languages, web frameworks, database engines, and IoT communication protocols:

Core Languages & Web Frameworks

- **Python 3.x:** Serves as the primary execution engine for telemetry simulation, thread management, asynchronous networking, and backend database interaction.
- **Flask Framework:** A micro web framework used to route HTTP endpoints, render server-side HTML templates, and deliver real-time telemetry JSON feeds to web browsers.
- **HTML5 & CSS3:** Provide structural layout and custom blue/grey visual styling for the responsive patient monitoring web dashboard.

Data Storage & Serialization

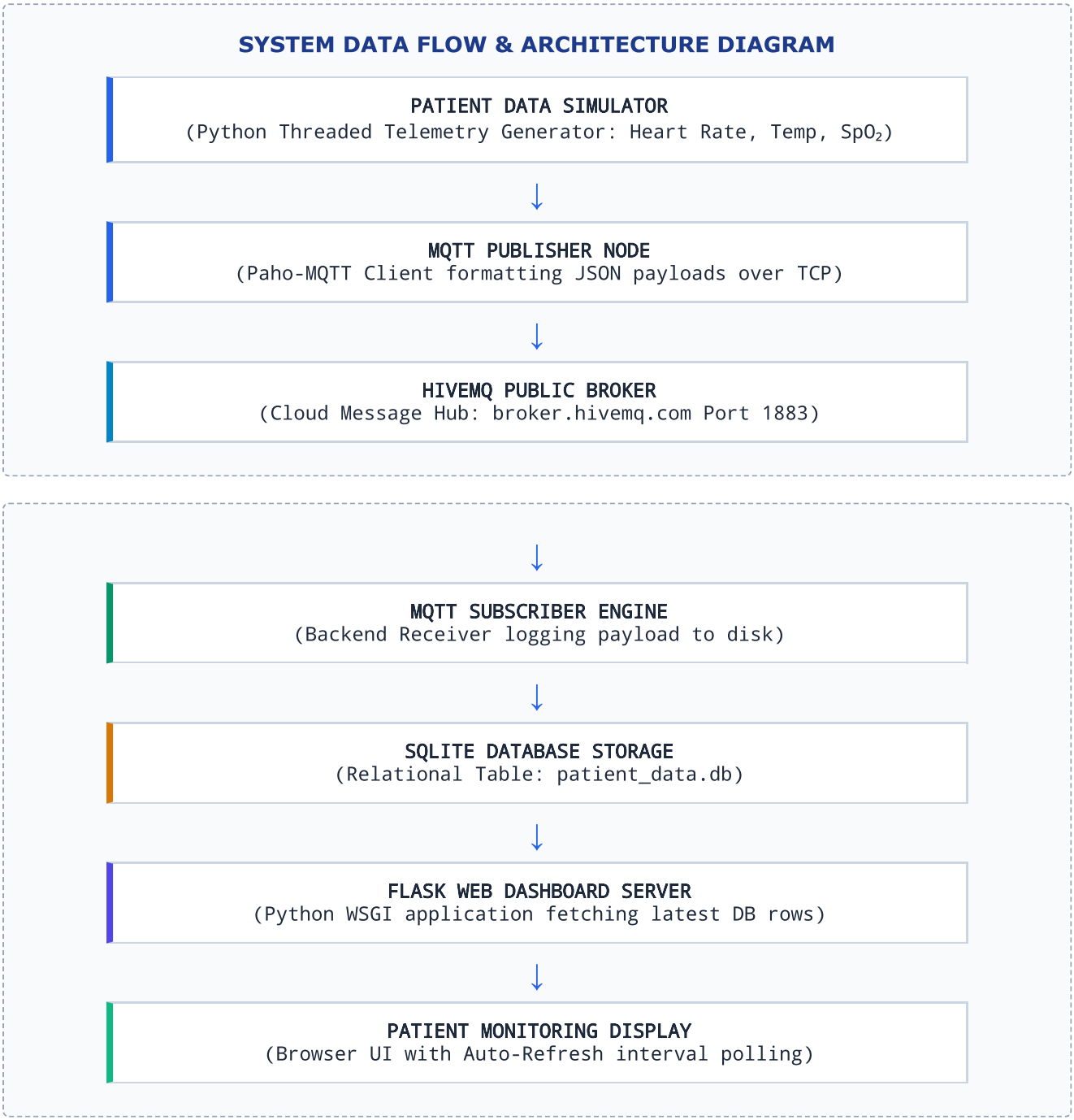
- **SQLite 3:** A lightweight, serverless relational database engine used to store historical patient readings, timestamped telemetry logs, and diagnostic states.
- **JSON (JavaScript Object Notation):** The standard payload format used across MQTT and CoAP protocol messages for clear key-value structure.

IoT Protocols & Networking Libraries

- **MQTT Protocol:** An ISO-standard (ISO/IEC 20922) publish-subscribe messaging protocol operating over TCP/IP, designed for low-bandwidth and high-latency environments.
- **Paho-MQTT Library:** An open-source Python client library handling connection keep-alives, topic subscription, and callback-driven payload ingestion.
- **HiveMQ Public Broker:** A public cloud-hosted MQTT broker (`broker.hivemq.com:1883`) acting as the centralized message hub between publishers and subscribers.
- **CoAP Protocol & aiocoap:** An asynchronous Python implementation of the Constrained Application Protocol (RFC 7252) running over UDP for lightweight binary communication.
- **Contiki-NG & Cooja Simulator:** An open-source operating system for low-power wireless IoT devices, paired with the Cooja network simulator to analyze IPv6 over Low-Power Wireless Personal Area Networks (6LoWPAN).

11. SYSTEM ARCHITECTURE

The complete software system is structured into a multi-tier decoupled pipeline. Simulated sensor threads publish JSON payloads to an external cloud broker, where an asynchronous backend process ingests the messages, updates the relational database, and feeds the web display.



ADDITIONAL NETWORK SIMULATION COMPONENTS: Running alongside the primary MQTT-Flask pipeline are experimental components: (1) *CoAP Server/Client* modules for UDP-based request-response testing, and (2) *Contiki-NG / Cooja* simulation environment for observing 6LoWPAN mesh network routing.

12. DETAILED SYSTEM MODULES

The overall software system is organized into five modular subsystems, each responsible for a distinct stage in the telemetry pipeline:

Module 1: Patient Telemetry Simulator

Generates realistic, continuous physiological readings for multiple virtual patients. It runs on a configurable timer loop and introduces small stochastic variations to simulate natural biological changes in Heart Rate (60–100 BPM), Body Temperature (36.5–37.5°C), and Blood Oxygen Saturation (95–100%).

Module 2: MQTT Communication Engine

Handles the network connection to the public HiveMQ MQTT broker. The publisher module serializes telemetry readings into JSON objects and sends them to designated topics (e.g., `healthcare/patient/+/telemetry`). The subscriber module stays connected, receives arriving messages asynchronously, and triggers persistence routines.

Module 3: Relational Persistence Layer (SQLite)

Provides robust local data storage using SQLite. It maintains schema structures, manages database connections, handles parameter binding to prevent injection vulnerabilities, and executes query functions to retrieve historical records for the web front-end.

Module 4: Flask Web Dashboard

Serves as the user interface layer for clinical operators. The Flask application renders web pages using HTML templates styled with custom CSS. It exposes REST API endpoints that return recent patient telemetry, enabling real-time dashboard updates via automatic browser polling.

Module 5: Experimental CoAP & Contiki-NG Simulator

A secondary network research module. It includes Python scripts based on *aiocoap* for UDP-based CoAP server testing, along with C-based Contiki-NG firmware files designed to run inside the Cooja simulation environment to analyze 6LoWPAN packet headers.

13. PATIENT DATA SIMULATION LOGIC

To emulate real medical telemetry without physical hardware, the system implements a stochastic generation algorithm. The simulator initializes a set of default patient profiles, assigning each a unique identifier, full name, age, and assigned room number.

Physiological Parameter Ranges

Parameter Name	Unit	Normal Range	Simulated Range	Generation Formula
Heart Rate (HR)	BPM	60 – 100	58 – 105	`base_hr + uniform(-3, 3)`
Body Temperature	°C	36.5 – 37.5	36.2 – 38.2	`base_temp + uniform(-0.2, 0.2)`
Oxygen Saturation (SpO_2)	%	95 – 100	93 – 100	`base_spo2 + uniform(-1, 1)`

JSON Payload Serialization Structure

Once parameters are calculated, the simulator packages the data into a standard JSON string format for network transmission:


```
{ "patient_id": "P-101", "patient_name": "Alice Smith", "timestamp": "2026-11-02 10:15:30",
"telemetry": { "heart_rate": 74, "temperature": 36.8, "spo2": 98 }, "status": "NORMAL" }
```


14. MQTT COMMUNICATION ARCHITECTURE

The Message Queuing Telemetry Transport (MQTT) protocol forms the core telemetry backbone of this system. Its lightweight pub/sub model makes it ideal for constrained networking scenarios.

DIAGRAM: MQTT Communication Workflow

Publisher Nodes → HiveMQ Public Broker (Port 1883) → Subscriber Service → SQLite Pipeline

Publish-Subscribe Broker Integration

The system connects to the public HiveMQ broker (`broker.hivemq.com`) on default port 1883. Using the Paho-MQTT Python library, the publisher loop generates telemetry readings every 3 seconds and broadcasts them across specific topic namespaces:

- `healthcare/sim/patient/P-101/data`
- `healthcare/sim/patient/P-102/data`
- `healthcare/sim/patient/P-103/data`

Quality of Service (QoS) Selection

The simulation uses **QoS Level 0 (At most once)** for routine telemetry updates to minimize network overhead and latency. In high-density network environments, this ensures that missing a single sensor reading does not block subsequent telemetry packets.

15. SQLITE DATABASE MANAGEMENT

Data persistence is managed through an embedded SQLite database engine (`patient_data.db`). The database schema is designed to store incoming JSON telemetry efficiently.

```
CREATE TABLE IF NOT EXISTS patient_telemetry ( id INTEGER PRIMARY KEY AUTOINCREMENT,
patient_id TEXT NOT NULL, patient_name TEXT NOT NULL, heart_rate INTEGER NOT NULL,
temperature REAL NOT NULL, spo2 INTEGER NOT NULL, recorded_at DATETIME DEFAULT
CURRENT_TIMESTAMP );
```

16. FLASK WEB DASHBOARD

The web front-end is implemented using the Flask WSGI micro-framework. The server exposes routes that fetch records from the SQLite database and display them on a clean, responsive web page.

PLACEHOLDER: Flask Web Dashboard Interface

Actual UI screenshot displaying active patient cards, vital sign metrics, and color-coded status badges.

Automatic Refresh Architecture

To provide real-time updates without page reloads, the dashboard interface uses JavaScript AJAX polling. Every 3 seconds, an asynchronous request fetches the latest telemetry from the backend `/api/v1/telemetry/latest` endpoint and dynamically updates the DOM elements.

17. COAP COMMUNICATION IMPLEMENTATION

In addition to TCP-based MQTT, the project includes an experimental implementation of the Constrained Application Protocol (CoAP, RFC 7252). Built using Python's `aiocoap` library, CoAP runs over UDP to reduce header overhead and connection handshake times in constrained networks.

The CoAP server initializes asynchronous resources under URI paths such as `coap://localhost/patient/telemetry`. While the CoAP server starts up successfully and binds to its UDP socket, complete end-to-end client fetching and validation are still under active development.

18. CONTIKI-NG AND COOJA NETWORK SIMULATION

To evaluate network routing behavior in low-power wireless networks (6LoWPAN), the project incorporates firmware templates built for the Contiki-NG operating system, designed for execution inside the Cooja simulation environment.

PLACEHOLDER: Cooja Network Simulator Environment

Visual network topology display showing simulated 6LoWPAN mote nodes, radio traffic visualizer, and mote output log terminals.

Within Cooja, mote nodes are configured to run 6LoWPAN stacks with RPL (Routing Protocol for Low-Power and Lossy Networks). Initial simulation runs confirm basic node boot sequences and radio message exchanges, though full UDP client-server routing remains a target for future testing.

19. SYSTEM TESTING & RESULTS SUMMARY

Rigorous operational testing was conducted across all core components of the project. The table below provides an honest summary of current testing results:

Subsystem / Feature	Technology Stack	Operational Status	Verification Details
Patient Data Simulator	Python / Random Engine	**FULLY TESTED**	Continuous background payload generation verified.
MQTT Publishing/Subscribing	Paho-MQTT / HiveMQ Broker	**FULLY TESTED**	Successful payload transmission over port 1883.
SQLite Persistence	SQLite3 / Python DB-API	**FULLY TESTED**	Reliable record insertion and history querying.
Flask Web Dashboard	Flask / HTML / AJAX	**FULLY TESTED**	Real-time UI updates via automated endpoint polling.
CoAP Server Setup	aiocoap / asyncio	**PARTIALLY TESTED**	Server starts and binds; client-side verification pending.
Contiki-NG / Cooja Simulation	Cooja / 6LoWPAN / IPv6	**PARTIALLY TESTED**	Motes boot and emit radio traffic; full UDP testing pending.

20. GITHUB REPOSITORY ORGANIZATION

The code is publicly maintained on GitHub at:
`https://github.com/srilekhasunil65-byte/Simulation_Healthcare_IoT`

- `sensor_simulator/`: Contains Python scripts for generating multi-patient telemetry streams.
- `app.py`: The primary Flask WSGI backend serving the UI and API endpoints.
- `mqtt_subscriber.py`: The background thread consuming MQTT messages and persisting them to SQLite.
- `coap_server.py`: Experimental `aiocoap` resource binding script.
- `docs/ & screenshots/`: Technical documentation, architectural flowcharts, and UI placeholders.

21. LIMITATIONS & FUTURE ENHANCEMENTS

While the project successfully demonstrates a full end-to-end software simulation pipeline, several current limitations point toward clear opportunities for future development:

- **Simulation-Only Data:** The system relies entirely on synthetic data generation, lacking hardware sensor interfaces (e.g., MAX30102, DHT11) or microcontroller integration (ESP32/Raspberry Pi).
- **Security & Encryption:** Transmissions currently use unencrypted MQTT (port 1883) without TLS/SSL wrappers, exposing payloads to potential network eavesdropping.
- **Incomplete Experimental Protocols:** CoAP client workflows and Contiki-NG 6LoWPAN routing require further end-to-end integration and automated test suites.

Planned Future Enhancements

1. Integrate hardware sensor inputs via ESP32 microcontrollers.
2. Implement MQTTS (MQTT over TLS) and authentication headers for enhanced security.
3. Complete the CoAP client-server integration and fully evaluate 6LoWPAN performance metrics.

22. CONCLUSION

The **Smart Healthcare Patient Monitoring System – IoT Simulation** project successfully establishes a robust, software-driven framework for modeling patient telemetry transmission, message queuing, relational storage, and real-time dashboard display. By decoupling data generation from visualization through standard MQTT pub/sub mechanisms and an SQLite/Flask pipeline, the system demonstrates an effective, scalable, and risk-free approach to IoT network prototyping.

23. REFERENCES

1. MQTT Version 3.1.1. OASIS Standard. Available online: mqtt.org
2. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
3. Shelby, Z., Hartke, K., & Bormann, C. (2014). *The Constrained Application Protocol (CoAP)*. RFC 7252, IETF.
4. Dunkels, A., et al. (2004). *Contiki - a lightweight and flexible operating system for tiny networked sensors*. IEEE LCN.